INTERNSHIP REPORT

Submitted

For the Partial Fulfilment of the Requirement for the Degree of Bachelor of _____ with _____ (Major and Minor Subjects) under **Magadh University**

Submitted to

The Department of _____, _____ **(College Name)**

[Paste College Logo here]

Submitted by

Name : _____

Semester : _____

Programme FYUGP : _____

University's Registration No. : _____

College Roll No. : _____

Year : _____

RK NEXORA
CAMPUS TO CAREER
A Unit of RKNexora Private Limited.

INTERNSHIP REPORT

Topic - Full Stack Web Development

Submitted to the Head of Department, _____ (College Name), in partial fulfilment of the requirements for the Degree of _____, Magadh University

Name of Student : _____

Programme : _____

University Registration No. : _____

College Roll No. : _____

Internship Registration No. : _____

Name of the IPO / Agency : RKNEXORA PRIVATE LIMITED

Internship Period : _____ to _____

Semester : _____

Year / Session : _____

RKNEXORA PRIVATE LIMITED
RKNEXORA
PRIVATE LIMITED
GAYA, BIHAR - 805236, INDIA

_____
Signature of Student

_____
Seal & Signature of IPO

ACKNOWLEDGEMENT

I express my sincere gratitude to RK NEXORA, RKNEXORA PRIVATE LIMITED, _____ (College Name), my faculty mentors and the internship supervisor, _____, for their guidance and support. The structured activities, learning resources and regular feedback helped me complete this internship in Full Stack Web Development successfully.

I am thankful to the Head of the Department and the college mentor for their encouragement and for guiding me in preparing this report. I am also grateful to the management of RKNEXORA PRIVATE LIMITED for providing a professional learning environment and the opportunity to learn and apply my knowledge in a practical setting. The concept sessions, hands-on tasks with sample data and case studies, and the supervisor's patient and detailed feedback helped me understand how a complete web application is built from the screen to the database.

Finally, I thank my family and peers for their constant encouragement throughout the programme.

_____

Name: _____

University Registration No. - _____

INTERNSHIP COMPLETION CERTIFICATE

The Internship Completion Certificate issued by RKNEXORA PRIVATE LIMITED for the internship in Full Stack Web Development is to be attached on this page.

[Attach the original / printed copy of the QR-verified Internship Completion Certificate here]

Certificate No. _____

CONTENTS

Acknowledgement..... 3

Internship Completion Certificate..... 4

Preface..... 6

Executive Summary..... 7

About the Agency..... 8

Plan and Execution..... 9

Learning Outcome..... 23

Conclusion..... 25

Attendance Sheet..... 27

PREFACE

Internships are an important part of the undergraduate programme, as they allow students to see how classroom learning is applied in real organisations. The Four-Year Undergraduate Programme (FYUGP), introduced in line with the National Education Policy 2020, gives special importance to experiential learning, skill development and employability. This report has been prepared as part of the internship requirement of Magadh University and the Higher Education guidelines applicable in Bihar.

The report describes the internship undertaken in Full Stack Web Development at RKNEXORA PRIVATE LIMITED, the plan followed, the work carried out, and the learning gained. A modern web application, whether it is an online shop, a railway booking site or a college portal, has three parts: the page the user sees (the front end), the program that processes requests (the back end) and the place where information is stored (the database). A full stack developer understands all three and can build a complete working application. This is one of the most sought-after skills in the technology industry. The report records how the front-end and back-end technologies were learnt and how a complete application with a login, a REST API and a database was built, run and tested during the internship. The real code, the real server output and screenshots of the running application are included.

The report has been arranged in the order suggested by the University, beginning with the cover page and acknowledgement and ending with the conclusion and attendance sheet. It is hoped that it gives a clear and honest account of the internship experience. All examples in this report use dummy data and fictitious names; no real personal or business information has been handled.

EXECUTIVE SUMMARY

This report documents the internship completed in Full Stack Web Development at RKNEXORA PRIVATE LIMITED (RK NEXORA - 'Campus to Career'). The programme required a minimum of 120 hours, completed over about four to five weeks (28 working days) with daily sessions of four to five hours, and was supported by an attendance log, supervisor guidance and review.

Background. Full stack web development means building both the client side and the server side of a web application. The front end is written with HTML, CSS and JavaScript and runs in the browser; the back end is written in a language such as JavaScript (Node.js) and runs on a server, where it receives requests, applies rules, checks who the user is and reads or writes data in a database. The two sides talk to each other through an API, usually in the JSON format. Understanding this whole path, from a button click to a row in a database and back, is the core of the subject.

Objectives. The internship aimed to (i) build web pages with HTML, CSS and JavaScript, (ii) make them responsive, (iii) understand the component idea of a front-end framework such as React, (iv) build a server with Node.js and write routes, (v) design a REST API with CRUD operations and test it, (vi) connect the application to a database and use SQL, (vii) understand authentication, password hashing and tokens, (viii) use Git and GitHub for version control, and (ix) build, test and document a small full stack application.

Work carried out. Each topic had a concept session followed by hands-on coding. The student first built pages with HTML, CSS and JavaScript, then wrote a server and an API, connected a database, added login and finally combined everything in one application. The code was run on a computer, the API was tested with the curl command and the real results are recorded in this report. The later part of the programme was devoted to the project, a case study exercise, a mock practical exercise, supervisor review and preparation of this report.

Project. The work was tied together in a project titled **Full Stack Web Application Project**. It is a small application called Study Task Tracker, in which a logged-in user can add, complete and delete study tasks. It has a front end in HTML, CSS and JavaScript, a back end written in Node.js with a REST API, a login protected by a password hash and a token, and an SQLite database. About 170 lines of code were written, and the version history was kept in Git.

Method. A blended method was followed - learn, practise, document and review - supported by a daily attendance log with day-wise remarks and regular feedback from the supervisor.

Learning gained. The student gained practical skill in front-end development, server-side programming with Node.js, designing and testing REST APIs, SQL databases, authentication with hashed passwords and tokens, version control with Git, and the habit of testing every layer of an application separately before joining them together.

Key observations. A full stack application is a chain of small parts and a fault in any link breaks the whole; testing each part separately (page, API, database) saves time; security must be thought of from the start and not added at the end; and version control makes it safe to experiment.

Conclusion. The internship met its objectives and contributed to the student's academic understanding, employability and confidence. The report follows the University's suggestive format: cover page, acknowledgement, certificate, contents, preface, executive summary, about the agency, plan and execution, learning outcome, conclusion and attendance sheet.

ABOUT THE AGENCY

RK NEXORA, a unit of RKNEXORA PRIVATE LIMITED, provides structured internship and skill-development programmes under the theme 'Campus to Career'. The organization focuses on academic-industry alignment, practical exposure, assessment, documentation and employability-oriented learning.

Particulars	Details
Name of Agency	RKNEXORA PRIVATE LIMITED
Programme / Unit	RK NEXORA - Campus to Career
Registered Office	C/O Mukesh Kumar, Khiri, PS Atri, Jethiyan, Atri, Gaya - 805236, Bihar, India
CIN	U85499BR2026PTC086757
Email	rknexorapvtltd@gmail.com
Website	www.rknexora.org
Internship Topic	Full Stack Web Development

1. Vision

To support students in developing practical, professional and industry-relevant capabilities.

2. Mission

To deliver structured learning experiences that connect academic knowledge with workplace expectations.

3. Activities of the Agency

1. Structured internship programmes aligned with university and higher-education requirements.
2. Skill-development learning modules, practical activities and assignments.
3. Attendance tracking, supervision, assessment and documentation.
4. Employability-oriented guidance connecting campus learning to career readiness.

4. Programme Structure

The internship programme is organised around a fixed number of learning hours, completed over a defined period. Each intern is registered on the internship portal with a unique registration number, is assigned a supervisor, follows a day-wise plan and maintains an attendance log. Learning is organised into three phases: **Orientation** (Days 1-2), **Learning and Practice** (Days 3-22, ten sub-topics of two days each: one concept day and one hands-on day) and **Project and Review** (Days 23-28), followed by a concluding review.

5. Role of the Supervisor

The internship supervisor assigned by the agency guided the student through the programme, explained concepts, assigned practice tasks, reviewed the daily log and gave feedback. The supervisor also verified attendance and reviewed the student's work at the end of the programme.

PLAN AND EXECUTION

1. Objectives

The internship was planned around the following objectives, each linked to a part of the programme:

1. Build web pages with HTML, CSS and JavaScript and make them responsive (Week 1).
2. Understand components, props and state in a front-end framework such as React (Week 2).
3. Build a server with Node.js and learn routes and middleware, the ideas on which Express is based (Week 2-3).
4. Design a REST API with CRUD operations and test it (Week 3).
5. Connect the application to a database and perform CRUD operations with SQL (Week 3-4).
6. Learn authentication - login, password hashing, sessions and tokens (Week 4).
7. Use Git and GitHub for version control (throughout).
8. Build, test and document a small full stack application (Weeks 4-5).

2. Methodology

The programme followed a blended method made up of four connected steps, repeated for each topic:

Learn. Each topic began with a concept session in which the supervisor explained the idea, its main terms and how it is used in real organisations. Notes were prepared in the student's own words.

Practise. The concepts were then applied in guided hands-on tasks - building pages with HTML, CSS and JavaScript, writing a small component-based user interface, creating routes in Node.js, designing endpoints for create, read, update and delete operations, testing endpoints with curl, writing SQL statements for a tasks table, hashing a password with a salt, protecting routes with a token, and committing and branching code in Git. Only sample data and fictitious examples were used, so that no real personal or business information was handled.

Document. Every day's work was recorded in the internship attendance log with a short remark, and sheets, files and screenshots were saved so that the work could be reviewed.

Review. The supervisor reviewed the work, pointed out errors and suggested improvements. Review days, a case study and a mock practical exercise helped to consolidate the learning.

3. Understanding Full Stack Web Development - Concepts Studied

3.1 The three layers of a web application

A full stack application is usually described in three layers. The presentation layer is the front end - the pages and scripts that run in the user's browser. The application layer is the back end - the program on the server that receives requests, applies business rules, checks permissions and prepares answers. The data layer is the database that stores information permanently. The layers communicate in a fixed way: the browser sends an HTTP request, the server runs the matching route, asks the database for data and sends back a response, normally in JSON, which the browser turns into what the user sees. The figure below shows the architecture used in the project.

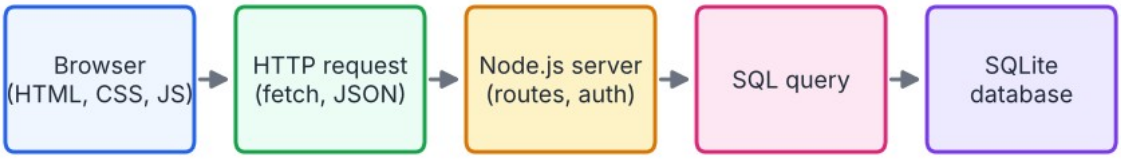


Figure 1: Architecture of the full stack application

3.2 Front end: HTML, CSS, JavaScript and component thinking

The front end was built with HTML for structure, CSS for style and JavaScript for behaviour, as in ordinary web development, with the fetch() function used to call the server without reloading the page. A framework such as React takes this further by dividing the page into small reusable components - for example a Task component - that receive data through props and remember their own data in state; when the state changes, the component redraws itself. The student practised these ideas and applied the same thinking in the project by writing small functions such as loadTasks(), toggle() and remove(), each responsible for one job.

3.3 Back end: Node.js, routes and middleware

Node.js lets JavaScript run on a server. A server listens on a port, and for each request it looks at the method (GET, POST, PUT, DELETE) and the path to decide what to do; this decision is called routing. Middleware is code that runs before the route, for example to read a JSON body or to check a login token. Frameworks such as Express make routing and middleware shorter to write, but they do the same work as the built-in http module. In this internship the routing and middleware logic were first practised with Express-style code and the project server was written with Node's built-in http module, so that it runs without installing any package; the same routes can be moved to Express with very little change.

3.4 REST API, database and authentication

A REST API exposes resources through URLs and uses the HTTP methods for the four basic operations, known as CRUD: POST creates, GET reads, PUT updates and DELETE removes. In the project the resource is the task, at /api/tasks. The data is stored in an SQL database table with columns for id, title and done status; SQL statements such as SELECT, INSERT, UPDATE and DELETE are used, and parameters are passed separately (the ? marks) to prevent SQL injection. Authentication proves who the user is: the password is never stored as plain text but as a salted hash, and after a correct login the server issues a random token that the browser sends with each later request. Without a valid token, the server answers 401 Unauthorised.

3.5 JSON, HTTP messages and the command line

Data travels between the browser and the server as text in the JSON format, in which values are written as name-value pairs. A request has a method, a path, headers (such as Content-Type and Authorization) and sometimes a body; a response has a status code, headers and a body. For example, the request to add a task carries the body shown below, and the server answers with the new task and the code 201. Working with the command line - starting the server, running curl, using Git - was part of the daily routine, because a developer must be comfortable outside the editor as well.

```
POST /api/tasks HTTP/1.1
Content-Type: application/json
Authorization: Bearer <token>

{ "title": "Revise HTML and CSS" }

--- response ---
HTTP/1.1 201 Created
{ "id": 1, "title": "Revise HTML and CSS", "done": 0 }
```

4. Week-wise Plan and Execution

Week	Days	Activities
Week 1	Days 1-6	Orientation and front-end foundations: company induction, HTML and CSS, JavaScript fundamentals and responsive design, each with a study session and a practice task.
Week 2	Days 7-12	Front-end framework ideas (components, props, state) and Node.js basics with routes and middleware.
Week 3	Days 13-18	REST API design and testing, and database integration with SQL.
Week 4	Days 19-24	Authentication, Git and GitHub, and the Study Task Tracker project; case study exercise and mock practical exercise.
Week 5	Days 25-28	Project closure: supervisor review and feedback, internship report drafting, presentation preparation and final review.

5. Week-wise Narrative

Week 1: The first week was devoted to orientation and foundations. After company induction, the student built pages with HTML and CSS, learnt JavaScript variables, functions, events and the DOM, and made the pages responsive with media queries.

Week 2: In the second week the student studied the component idea of a front-end framework - components, props and state - and then moved to the server side, learning how Node.js listens for requests, how routes are written and what middleware does.

Week 3: The third week was devoted to the REST API, in which endpoints for create, read, update and delete were designed and tested with curl, and to the database, where SQL statements were written and connected to the server.

Week 4: In the fourth week the student studied authentication - password hashing, tokens and protected routes - and Git and GitHub. The Study Task Tracker application was assembled, tested, and a case study and a mock practical exercise were completed.

Week 5: The last week was devoted to project closure: supervisor review and feedback, drafting of the internship report, preparation of a presentation and final review before submission.

6. Topic-wise Learning in Detail

6.1 HTML and CSS

Concept: Page structure with semantic tags, forms, selectors, the box model and layouts with Flexbox. **Practice carried out:** Building a login page and a task list page with a clean card layout.

6.2 JavaScript Fundamentals

Concept: Variables, functions, events, arrays, objects, JSON and the DOM. **Practice carried out:** Writing functions that read input, update the page, and a to-do list that adds and removes items.

6.3 Responsive Design

Concept: Viewport, media queries and flexible layouts. **Practice carried out:** Testing the pages at phone and desktop widths and fixing overflow problems.

6.4 React and Front-end Frameworks

Concept: Components, props, state and building a user interface from small pieces. **Practice carried out:** Designing a Task component with a title and a Done button, and planning how state changes redraw the list.

6.5 Node.js and Express

Concept: Server setup, ports, routes, middleware, request and response objects. **Practice carried out:** Writing a server that returns JSON for GET requests and reads the body of POST requests.

6.6 REST API Development

Concept: Endpoints with CRUD operations, HTTP methods, status codes and testing. **Practice carried out:** Designing /api/tasks endpoints and testing them with curl, including wrong inputs (section 9).

6.7 Database Integration

Concept: Tables, SQL statements, parameters and connecting an application with a database. **Practice carried out:** Creating the tasks table in SQLite and writing SELECT, INSERT, UPDATE and DELETE statements from the server.

6.8 Authentication Basics

Concept: Login, sessions, password hashing, salt and tokens. **Practice carried out:** Hashing a password with scrypt, issuing a random token and protecting the POST, PUT and DELETE routes.

6.9 Git and GitHub

Concept: Repositories, commits, branches, merges and pull requests. **Practice carried out:** Making commits for each part of the project, creating a branch for a feature and merging it (Figure 7).

6.10 Mini Full Stack Project

Concept: Building a small app with front end, back end and database. **Practice carried out:** Building the Study Task Tracker described in section 10.

7. Good Practice, Security and Ethics - Points Studied

A full stack developer handles other people's data, so security and responsibility are part of the job. The points studied are summarised below.

- **Never store plain passwords:** store only a salted hash, as done in the project with scrypt.
- **Validate on the server:** the browser can be bypassed, so every input must be checked again on the server (the title length check in the API).
- **Use parameterised SQL:** passing values with ? prevents SQL injection.
- **Protect routes:** actions that change data need a valid token; wrong or missing tokens receive 401.

- **Use HTTPS and keep secrets out of code:** passwords, keys and tokens should be kept in environment variables and never committed to Git (see .gitignore).
- **Use the right status codes:** 200, 201, 400, 401 and 404 tell the client exactly what happened.
- **Respect privacy and law:** collect only the necessary data; the Digital Personal Data Protection Act, 2023 applies to personal data.
- **Version control and honesty:** commit often with clear messages, and acknowledge code that is taken from documentation or open-source libraries.

8. Process Flows Studied

8.1 Request-response flow of the application

- The user clicks a button; the JavaScript calls fetch() with the URL, method and token.
- The server receives the request and the route is chosen from the method and the path.
- Middleware reads the JSON body and the token is checked for protected routes.
- The route runs an SQL statement on the database.
- The server sends a JSON response with a status code; the browser updates the page.

8.2 Feature development with Git

- Create a branch for the feature (git checkout -b feature-name).
- Write and test the code; commit with a clear message.
- Merge the branch into the main branch after the test passes.
- Push to GitHub and, in a team, raise a pull request for review.

9. Worked Examples with Sample (Dummy) Data

Worked example - SQL statements for the tasks table. The four basic SQL statements used by the server are shown below with the meaning of each. The question marks are filled in by the program, which keeps user input separate from the SQL text.

```
CREATE TABLE IF NOT EXISTS tasks (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  title TEXT NOT NULL,
  done INTEGER DEFAULT 0
);
INSERT INTO tasks (title) VALUES ('Revise HTML and CSS');
SELECT * FROM tasks ORDER BY id;
UPDATE tasks SET done = 1 - done WHERE id = 1;   -- toggles 0 and 1
DELETE FROM tasks WHERE id = 3;
```

The expression 1 - done flips the value: if done is 0 the result is 1, and if done is 1 the result is 0, so one PUT request can both tick and untick a task. The WHERE clause is important - an UPDATE or DELETE without it would change every row of the table.

Worked example - password hashing. A small script was written to understand why passwords are never stored in plain form. The script hashes a password with scrypt and a salt and compares the result for the right and a wrong password.

```
const crypto = require("node:crypto");
const salt = "demo-salt";
const hash = (pw) => crypto.scryptSync(pw, salt, 32).toString("hex");

const stored = hash("learn123");
console.log("Stored hash   :", stored.slice(0, 32) + "...");
console.log("Right password:", hash("learn123") === stored);
console.log("Wrong password:", hash("learn124") === stored);
```

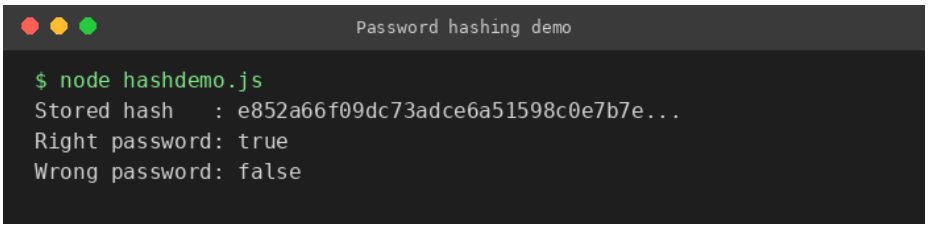


Figure 2: Output of hashdemo.js

Only the long hexadecimal hash is stored. When a user logs in, the typed password is hashed again and the two hashes are compared; a change of even one character gives a completely different hash. If the database is stolen, the attackers do not get the passwords themselves. The HTTP status codes met in the project are summarised in the table below.

Code	Meaning	Where it appeared in the project
200 OK	Request succeeded	Listing tasks, login, toggling a task
201 Created	A new resource was created	Adding a task
400 Bad Request	The client sent wrong data	Task title shorter than three letters
401 Unauthorised	Login missing or wrong	POST without a token, wrong password
404 Not Found	The URL does not exist	Unknown file or route
500 Server Error	A fault on the server	Not seen in testing, but must be handled

Worked example - testing the REST API. The API was tested one endpoint at a time with the curl command, before the front end was connected. The tests covered a request without login, a wrong password, a correct login, adding tasks, a too-short title and marking a task as done. The real output is shown below.

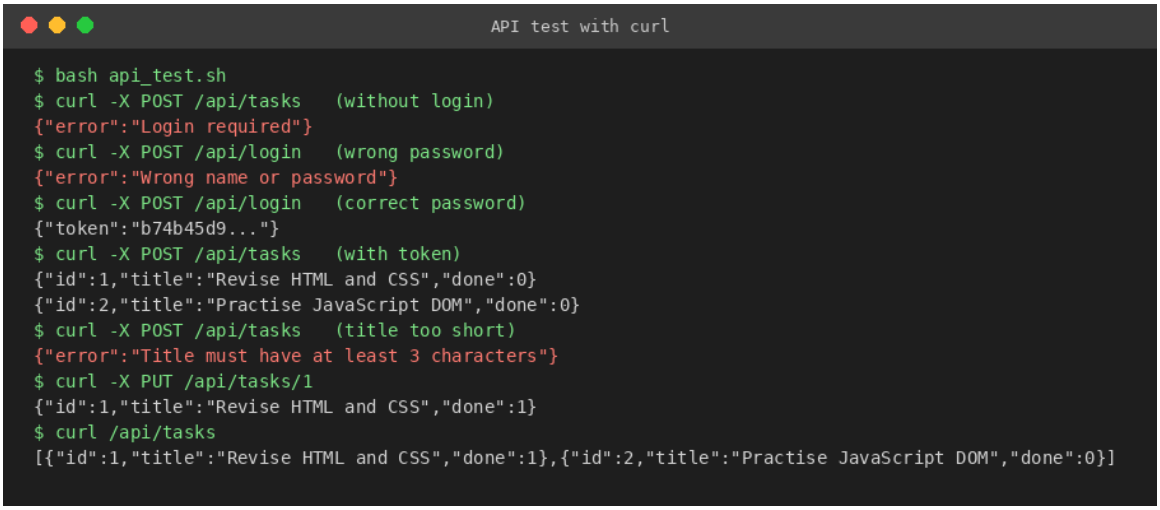


Figure 3: Testing the API with curl - real responses of the server

The server correctly refused the request without a token (401), refused the wrong password, issued a token after the correct login, created two tasks and answered 'Title must have at least 3 characters' for the title 'ab'. The PUT request changed the done value of task 1 from 0 to 1. The response codes used by the API are summarised below.

Method and path	Action	Login needed	Success response
GET /api/tasks	List all tasks	No	200 and a list
POST /api/login	Check password, issue token	No	200 and token
POST /api/tasks	Add a task	Yes	201 and the new task
PUT /api/tasks/:id	Toggle done	Yes	200 and the task
DELETE /api/tasks/:id	Delete a task	Yes	200 and the id

10. Project: Full Stack Web Application Project

Problem statement. Students often write their study tasks on scraps of paper and forget them. The aim of the project was to build a small web application in which a student can log in, add study tasks, mark them as done and delete them, with the tasks stored safely in a database. The user name, password and tasks are dummy data.

10.1 Requirements and design

- A login page; wrong credentials must show an error.
- After login, a form to add a task (at least three letters) and a list showing all tasks.
- Buttons to mark a task as done (shown with a strike-through) and to delete it.
- A REST API with GET, POST, PUT and DELETE for tasks; changes allowed only for a logged-in user.
- Data stored permanently in an SQLite database; the password stored only as a hash.
- The code kept in a Git repository with meaningful commits.

The project has four files: server.js (back end), and index.html, style.css and app.js in a public folder (front end). The server serves the pages and the API from the same address, so no extra settings for cross-origin requests are needed.

10.2 Back-end code - server.js

The server creates the table if it does not exist, defines helper functions for sending JSON and reading the body, and then routes each request: files for the front end, /api/login for authentication and /api/tasks for the CRUD operations.

```
// Study Task Tracker - a small full stack app (Node.js http + SQLite)
const http = require("node:http");
const fs = require("node:fs");
const path = require("node:path");
const crypto = require("node:crypto");
const { DatabaseSync } = require("node:sqlite");

const db = new DatabaseSync("tasks.db");
db.exec(`CREATE TABLE IF NOT EXISTS tasks (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  title TEXT NOT NULL,
  done INTEGER DEFAULT 0
)`);

// Demo user: the password is stored only as a salted hash
const salt = "demo-salt";
const USER = { name: "intern", hash: crypto.scryptSync("learn123", salt,
32).toString("hex") };
const tokens = new Set();

function send(res, status, data) {
  res.writeHead(status, { "Content-Type": "application/json" });
  res.end(JSON.stringify(data));
}

function readBody(req) {
  return new Promise((resolve) => {
    let body = "";
    req.on("data", (chunk) => (body += chunk));
    req.on("end", () => { try { resolve(JSON.parse(body || "{}")); } catch {
resolve(null); } });
  });
}

function authorised(req) {
  const token = (req.headers.authorization || "").replace("Bearer ", "");
  return tokens.has(token);
}

const server = http.createServer(async (req, res) => {
  const url = new URL(req.url, "http://localhost");
  const parts = url.pathname.split("/").filter(Boolean);       // e.g.
["api","tasks","3"]

  if (parts[0] !== "api") {                                      // serve the frontend
    const file = path.join("public", url.pathname === "/" ? "index.html" :
url.pathname);
    if (!fs.existsSync(file)) return send(res, 404, { error: "Not found" });
    const type = file.endsWith(".js") ? "text/javascript" : file.endsWith(".css") ?
"text/css" : "text/html";
    res.writeHead(200, { "Content-Type": type });
    return res.end(fs.readFileSync(file));
  }

  if (parts[1] === "login" && req.method === "POST") {
    const { name, password } = (await readBody(req)) || {};
    const hash = crypto.scryptSync(String(password), salt, 32).toString("hex");
    if (name === USER.name && hash === USER.hash) {
```

```
      const token = crypto.randomBytes(16).toString("hex");
      tokens.add(token);
      return send(res, 200, { token });
    }
    return send(res, 401, { error: "Wrong name or password" });
  }

  if (parts[1] === "tasks") {
    const id = parts[2];
    if (req.method === "GET") {
      return send(res, 200, db.prepare("SELECT * FROM tasks ORDER BY id").all());
    }
    if (!authorised(req)) return send(res, 401, { error: "Login required" });

    if (req.method === "POST") {
      const body = await readBody(req);
      if (!body || !body.title || body.title.trim().length < 3)
        return send(res, 400, { error: "Title must have at least 3 characters" });
      const r = db.prepare("INSERT INTO tasks (title) VALUES
(?)").run(body.title.trim());
      return send(res, 201, { id: Number(r.lastInsertRowid), title: body.title.trim(),
done: 0 });
    }
    if (req.method === "PUT" && id) {
      db.prepare("UPDATE tasks SET done = 1 - done WHERE id = ?").run(id);
      return send(res, 200, db.prepare("SELECT * FROM tasks WHERE id = ?").get(id));
    }
    if (req.method === "DELETE" && id) {
      db.prepare("DELETE FROM tasks WHERE id = ?").run(id);
      return send(res, 200, { deleted: Number(id) });
    }
  }
  send(res, 404, { error: "Not found" });
});

server.listen(3000, () => console.log("Server running at http://localhost:3000"));
// title validation: minimum 3 characters
```

10.3 Front-end code - index.html and app.js

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Study Task Tracker</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <main>
    <h1>Study Task Tracker</h1>
    <section id="loginBox">
      <input id="user" placeholder="Name" value="intern">
      <input id="pass" type="password" placeholder="Password">
      <button id="loginBtn">Login</button>
      <p id="msg"></p>
    </section>
    <section id="app" hidden>
      <form id="addForm">
        <input id="title" placeholder="New task (min 3 letters)">
        <button>Add</button>
      </form>
      <ul id="list"></ul>
    </section>
  </main>
  <script src="app.js"></script>
</body>
</html>
```

The script keeps the token in a variable after login, sends it in the Authorization header with every API call, and redraws the list after each change.

```
let token = "";
const $ = (id) => document.getElementById(id);

async function api(url, method = "GET", body) {
  const res = await fetch(url, {
    method,
    headers: { "Content-Type": "application/json", Authorization: "Bearer " + token },
    body: body ? JSON.stringify(body) : undefined,
  });
  return { ok: res.ok, data: await res.json() };
}

async function loadTasks() {
  const { data } = await api("/api/tasks");
  $("list").innerHTML = "";
  data.forEach((t) => {
    const li = document.createElement("li");
    li.className = t.done ? "done" : "";
    li.innerHTML = `<span>${t.title}</span><span>
      <button class="tick" onclick="toggle(${t.id})">Done</button>
      <button onclick="remove(${t.id})">Delete</button></span>`;
    $("list").appendChild(li);
  });
}

async function toggle(id) { await api("/api/tasks/" + id, "PUT"); loadTasks(); }
async function remove(id) { await api("/api/tasks/" + id, "DELETE"); loadTasks(); }

$("loginBtn").onclick = async () => {
  const { ok, data } = await api("/api/login", "POST", { name: $("user").value,
password: $("pass").value });
  if (!ok) { $("msg").textContent = data.error; return; }
  token = data.token;
  $("loginBox").hidden = true;
  $("app").hidden = false;
  loadTasks();
};

$("addForm").onsubmit = async (e) => {
  e.preventDefault();
  await api("/api/tasks", "POST", { title: $("title").value });
  $("title").value = "";
  loadTasks();
};
```

10.4 Running the application

The server was started with the command node server.js and the application was opened at localhost:3000. A wrong password showed the error message; after a correct login, the task list was shown. A new task was added, and the second task was marked as done.

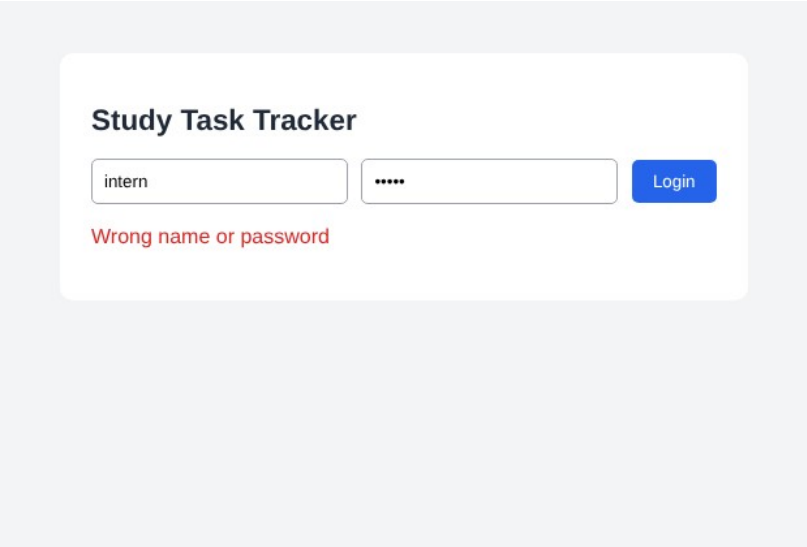


Figure 4: Login with a wrong password shows an error

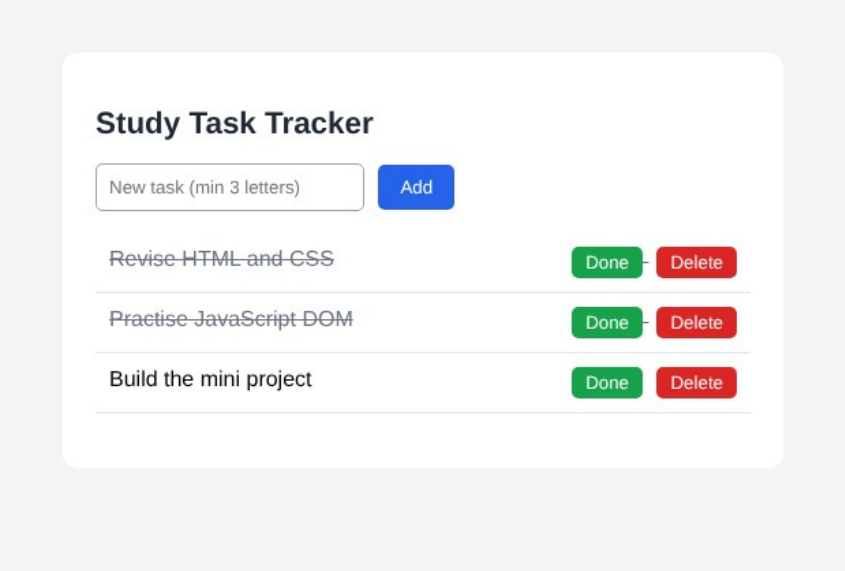


Figure 5: The application after login - three tasks, two marked done

10.5 Database check

The database file was then checked directly with an SQL query, which confirmed that the data shown in the browser was really stored in the database, with done = 1 for the two completed tasks.

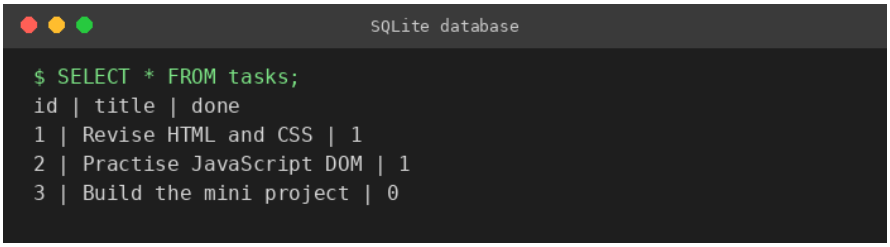


Figure 6: Contents of the tasks table in SQLite

10.6 Version control

The project was kept in a Git repository. The server, the front-end pages and the script were committed separately, a branch was created for a change and merged back, and the history is shown below.

Git history

```
$ git log --oneline --graph
*   e3d4430 Merge feature-validation
|\
| * 3b7964c Document title validation rule
|/
* 158357c Add app.js - login, add, done and delete tasks
* 6c3861c Add frontend page and styles
* aab09f9 Add server with SQLite and tasks API
```

Figure 7: Git history of the project

10.7 Limitations and future improvements

- Tokens are kept in the server's memory and are lost when it restarts; a database table or JSON Web Tokens with an expiry time can be used.
- There is only one demo user; registration, and tasks for each user, can be added.
- The front end uses plain JavaScript; it can be rebuilt with React components.
- The server can be moved to Express and deployed on a cloud service with HTTPS.
- Automated tests can be written for the API.

11. Case Study Exercise: An Online Notice and Admission Enquiry System for a Coaching Centre

A fictitious coaching centre in Patna receives admission enquiries by phone and WhatsApp. The staff write the names and numbers in a notebook, often forget to call back, and cannot say how many enquiries came in a week. The centre also puts up notices on a wall, which many students miss. The student was asked to suggest a full stack solution.

Analysis. There are two groups of users: visitors, who send an enquiry, and staff, who must read and update enquiries and publish notices. This needs a public page (front end), a protected staff page, an API and a database.

Solution proposed. A responsive website with an enquiry form (name, mobile number, course); a back end that validates the data and stores it in a database; a staff login with hashed passwords and tokens; a staff page that lists enquiries, marks them as 'called' and exports them; and a notice section that students can read on their phones. The pattern is the same as the Study Task Tracker: a table, a REST API, validation and login.

Expected benefit and cautions. No enquiry would be lost, call-backs could be tracked and notices would reach every student. The cautions noted were to protect the personal data of students and parents, to take backups of the database, to keep staff passwords strong and to inform visitors how their data will be used.

12. Mock Practical Exercise: Building and Testing a Small Full Stack Feature

A mock practical exercise was conducted at the end of the learning phase to check that the student could add a feature to a full stack application independently. The task was to add a 'priority' field (low, medium or high) to every task and show it in the list. The steps followed were:

1. Understanding the task: the changes needed in the database, API and page were listed.
2. Database: a new column priority was added to the tasks table with a default value.
3. API: the POST route was changed to accept and validate the priority; the GET route returned it.
4. Front end: a drop-down was added to the form and the priority was shown next to each title.
5. Testing with curl: valid and invalid priorities were sent and the status codes checked.
6. Testing in the browser: tasks were added with each priority and the list was checked.
7. Git: the work was done on a new branch and merged after testing, with clear commit messages.
8. Review: the supervisor checked the code structure and security, and suggested validating the priority against a fixed list, which was done.

13. Sample Checklists Prepared

API checklist

- Every endpoint uses the correct method and returns the correct status code.
- Inputs are validated on the server; error messages are clear.
- Routes that change data need a valid token.
- SQL statements use parameters; no password or secret is written in the response.

Application checklist

- Front end handles errors from the server and shows a message.
- Pages work on a phone width; no console errors.
- Passwords are stored as hashes; secrets are not committed to Git.
- The repository has clear commit messages and a .gitignore file.

14. Tools and Resources

The following tools and resources were used during the internship: Node.js (version 22) with its built-in http, crypto and SQLite modules, a code editor (Visual Studio Code), the Chrome browser and its developer tools, the curl command for testing APIs, Git for version control, GitHub for hosting repositories, Playwright for taking screenshots, MS Word for the report, MDN Web Docs and Node.js documentation, learning material given by the agency, and regular mentor feedback.

15. Programme at a Glance

The programme was in the domain of Full Stack Web Development with RKNEXORA PRIVATE LIMITED as the agency. It required a minimum of 120 hours over 28 learning days, organised in three phases: Orientation (Days 1-2), Learning and Practice (Days 3-22) and Project and Review (Days 23-28). Attendance was recorded in the Internship Attendance Log, signed by the supervisor (see Attendance Sheet).

The approximate distribution of the effort across the topics is shown below.

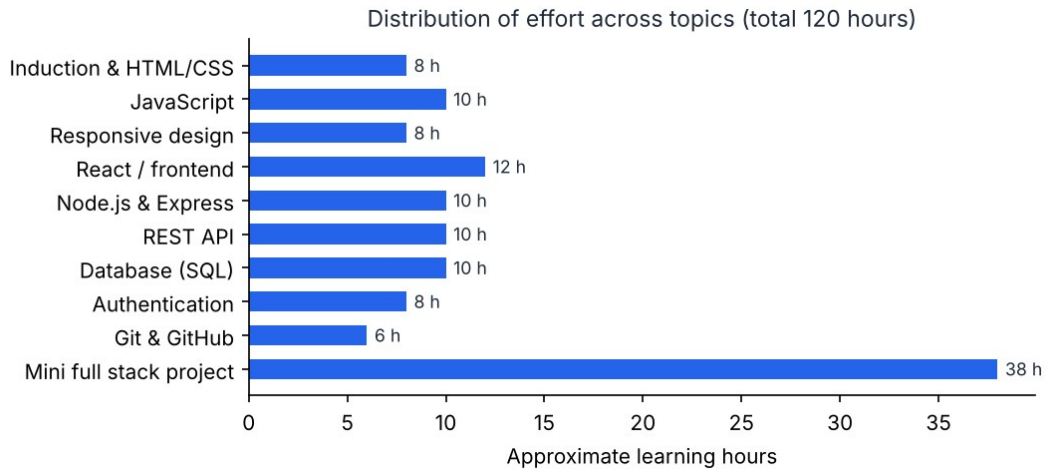


Figure 8: Distribution of learning effort across topics (120 hours)

LEARNING OUTCOME

a. Knowledge Acquired

1. The three layers of a web application and how a request travels from the browser to the database and back.
2. HTML, CSS, JavaScript, the DOM, fetch() and JSON; responsive design.
3. Component, props and state - the thinking of front-end frameworks such as React.
4. Node.js servers, ports, routing and middleware; the relation of Express to the built-in http module.
5. REST principles, HTTP methods, status codes and CRUD endpoints; testing APIs with curl.
6. SQL databases: tables, INSERT, SELECT, UPDATE and DELETE, and parameterised queries.
7. Authentication: salted password hashing, tokens, protected routes and the meaning of 401 and 400 responses.
8. Version control with Git: commits, branches, merging and GitHub.
9. Planning, building, testing and documenting a complete small application.

Glossary of Terms Learned

- **Front end / Back end** - The part running in the browser / the part running on the server.
- **API** - A set of URLs through which programs talk to each other.
- **REST** - A style of API design using resources and HTTP methods.
- **CRUD** - Create, Read, Update, Delete.
- **JSON** - A simple text format for data, such as {"id": 1}.
- **Route** - A rule linking a method and path to a piece of code.
- **Middleware** - Code that runs before the route to read or check the request.
- **Port** - A numbered door on a computer where a server listens.
- **Database / SQL** - Organised storage of data / the language to query it.
- **SQL injection** - An attack that puts harmful SQL into an input.
- **Hash / Salt** - A one-way scrambled form of a password / a random value added before hashing.
- **Token** - A random code proving that the user has logged in.
- **Status code** - A number in the response showing the result, such as 200 or 404.
- **Component / Props / State** - A reusable piece of the interface / its inputs / its changing data.
- **Git / Branch / Merge** - Version control tool / a separate line of work / joining branches.
- **Repository** - A folder tracked by Git.
- **Environment variable** - A setting kept outside the code, such as a secret key.
- **Deployment** - Putting an application on a server for users.

b. Skills Learned

Technical skills. Writing pages with HTML, CSS and JavaScript; calling an API with fetch(); building a Node.js server with routes and middleware; designing and testing REST endpoints; writing SQL statements and using parameters; hashing passwords and protecting routes with tokens; using Git for commits, branches and merges; and testing an application layer by layer.

Professional skills. Logical and structured thinking; problem solving and debugging across several layers; attention to security and detail; time management; clear documentation; and the ability to explain how an application works from the screen to the database.

These skills were developed through repeated practice and daily documentation. Testing each layer separately built a methodical way of finding faults, writing the API tests built accuracy, using Git built discipline, and explaining the project to the supervisor built confidence.

c. Observed Attitude and Gained Values

1. **Responsibility for user data:** passwords and personal data were treated carefully, and only dummy data was used.
2. **Patience in debugging:** a fault could be in the page, the request, the server or the database, and finding it needed calm, step-by-step checking.
3. **Honesty:** code from documentation and libraries was acknowledged, and what was not built (such as Express in the project) was stated plainly.
4. **Attention to security:** validation, hashing and tokens became habits.
5. **Discipline and punctuality:** regular reporting at fixed timings and daily attendance records.

d. Challenges Faced

Understanding how the front end and back end talk to each other was handled by testing the API with curl first and connecting the page only after the API worked.

Understanding asynchronous JavaScript (async and await) was difficult at first; it was handled by writing small examples of fetch() and printing each step.

Finding why a request returned 401 was handled by printing the Authorization header and checking the token on the server.

Writing SQL with parameters instead of joining strings was handled by studying SQL injection examples and always using the ? form.

Understanding Git branches and merging was handled by practising on a small repository and viewing the history graph.

Managing time between coding, documentation and report writing was handled by dividing work week by week and using review days for corrections.

e. Career Pathways

The knowledge gained supports careers such as Junior Full Stack Developer, Front-end Developer, Back-end or API Developer, Web Application Developer, Software Trainee, QA and Automation Tester, and, with further study, DevOps or Cloud Engineer. Opportunities exist in IT services and product companies, start-ups, e-commerce, fintech, education technology and government digital projects. Related higher-study options include BCA, MCA, B.Tech in computer science and information technology and certificate courses in full stack development, cloud computing and databases. A good next step is to learn React and Express in depth, learn one more database such as PostgreSQL or MongoDB, deploy a project online, and keep three or four projects on GitHub as a portfolio.

CONCLUSION

The internship in Full Stack Web Development was a valuable learning experience that integrated academic knowledge with practical activity. The programme achieved its objectives through structured learning, guided practice, assignments, supervisor review and documentation.

The student gained a clear understanding of how a complete web application is built - through the front end, the Node.js server, the REST API, the SQL database, authentication and version control - and acquired practical experience by building, running and testing the Study Task Tracker. The experience has strengthened the student's employability and provides a foundation for further study and work in software development.

Reflection

Looking back, the most useful lesson of the internship was that a web application is a chain of simple parts. When the page, the API and the database were each tested alone, a complicated application became easy to understand and to repair. Seeing a click in the browser change a row in the database was the moment when the whole picture became clear. The importance of security - hashing passwords, validating input and protecting routes - was understood through practice, and the habit of committing code with clear messages made the work safe and organised.

Objectives and Achievement

The objective of building pages and making them responsive was achieved through the login and task pages. The objective of understanding components and the Node.js server was achieved through the task component design and the server with routes and middleware. The objective of REST and databases was achieved through the CRUD API and the SQLite table, tested with curl. The objective of authentication was achieved through the salted hash and the token-protected routes. The objective of version control was achieved through the Git history, and the objective of building a complete application was achieved through the Study Task Tracker.

Future Scope

The knowledge gained can support further study of software engineering, cloud computing and data engineering, and careers in web and application development. The project can be extended with user registration, React components, a cloud database and deployment. With the growth of digital services, e-commerce and e-governance in Bihar and across India, developers who can build complete applications are increasingly needed.

Suggestions for Improvement of the Programme

1. Include a short module on deploying an application to a cloud service with HTTPS.
2. Add a team project in which the front end and back end are built by different members and joined through an API.
3. Provide practice with one more database such as PostgreSQL or MongoDB.
4. Continue regular mentor feedback and progress reviews.
5. Encourage students to maintain detailed daily documentation.

References

1. Kolb, D. A. (1984). Experiential Learning: Experience as the Source of Learning and Development. Prentice Hall.
2. University Grants Commission (UGC). Internship guidelines for higher education institutions.
3. Ministry of Education, Government of India. National Education Policy 2020.

4. Mozilla Developer Network (MDN). MDN Web Docs - JavaScript, HTTP and Web APIs. developer.mozilla.org.
5. OpenJS Foundation. Node.js Documentation. nodejs.org.
6. Fielding, R. T. (2000). Architectural Styles and the Design of Network-based Software Architectures (REST). Doctoral dissertation, University of California, Irvine.
7. SQLite Consortium. SQLite Documentation. sqlite.org.
8. Chacon, S. and Straub, B. Pro Git. Apress.
9. OWASP Foundation. OWASP Top Ten Web Application Security Risks. owasp.org.
10. Government of India. The Digital Personal Data Protection Act, 2023.
11. RK NEXORA internship programme handbook.

ATTENDANCE SHEET

The Internship Attendance Log issued by the IPO, with the supervisor's signature for each day, is to be attached on this page.

[Attach the signed Internship Attendance Log here]